

Reviewer Pack — Additive Energy of Out-Degree Sequence Bounds $\text{ACC}^0[2] \text{ MOD}_3$...

Entry 7e5a5fe0905f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 11:55:20 UTC

Additive Energy of Out-Degree Sequence Bounds $\text{ACC}^0[2] \text{ MOD}_3$ Size

Entry ID: 7e5a5fe0905f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 11:55:20 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics — additive energy $E(A) = \#\{(a,b,c,d) \in A^4 : a+b=c+d\}$, the second moment of the representation function $r_{\{A+A\}}$, in the Plünnecke–Ruzsa / Balog–Szemerédi–Gowers / Gowers–U² tradition. Applied here to the canonical out-degree multiset of the circuit DAG — a STRUCTURAL invariant of the wiring (not of the truth table; two $\text{ACC}^0[2]$ circuits computing the same function can have very different E), shielded from Razborov–Rudich since $E(C)$ is not a poly-time property of truth tables. The functional uses integer equality and counting only (no F_q polynomial extension of E exists since the count of integer additive quadruples is not preserved under polynomial-ring lifts), Aaronson–Wigderson algebrization-safe. Distinct from blacklisted Plünnecke doubling (which used $|A+A|/|A|$ on truth-table top-Fourier supports), Chang spectrum on wire-source labels (η -spectrum cardinality, not L^2 energy), and Mostar/Möbius/Birkhoff structural circuit attempts; an arXiv search ‘additive energy’ AND (‘ ACC^0 ’ OR ‘Williams algorithmic method’ OR ‘circuit lower bound’) returns 0 direct hits with <5 adjacent papers (all on additive-combinatorics matrix-rigidity bounds, never on ACC^0 circuit DAGs). **Field B** (complexity object): $\text{ACC}^0[2]$ circuit size $s(C)$ for the explicit function MOD_3 on n inputs ($q=3$ coprime to $m=2$), in the Williams 2011 / Murray–Williams 2018 algorithmic-method regime targeting $\text{NEXP} \not\subseteq \text{ACC}^0$ and explicit-function $\text{ACC}^0[2]$ lower bounds; the canonical Cook–Reckhow–Krajíček bounded-arithmetic stepping stone $V^0[2] \sqsubset V^0[2, \oplus_3]$ corresponds to this circuit-class separation.

Statement:

Let C be an $\text{ACC}^0[2]$ circuit (gates $\in \{\text{AND}, \text{OR}, \text{NOT}, \text{MOD}_2\}$, unbounded fan-in) of size s computing a Boolean function f_C on n inputs. Order the gates $g_1, \dots, g_s$ in canonical topological order (key: depth, then gate-type $\in \{\text{AND}, \text{OR}, \text{NOT}, \text{MOD}_2\}$, then sorted tuple of input-gate indices), and let $d_i :=$ out-degree of g_i in the DAG; let $E(C) := \#\{(i,j,k,l) \in [s]^4 : d_i + d_j = d_k + d_l\} / s^3$ be the normalized additive energy of the out-degree multiset. Conjecture: there exist absolute constants $c_1, c_2 > 0$ such that (i) every $\text{ACC}^0[2]$ circuit C with $\text{corr}(f_C, \text{MOD}_3) \geq 0.9$ (Boolean correlation over uniform inputs) and $s \geq 8$ satisfies $E(C) \geq c_1 / \log^2(s+1)$, while (ii) a uniformly random $\text{ACC}^0[2]$ circuit C' (same gate-type distribution and same s , but Erdős–Rényi-random wiring) satisfies $E(C') \leq c_2 \cdot s^{-1/2} \cdot \log(s+1)$ with probability ≥ 0.9 . A single $\text{ACC}^0[2]$ circuit C with $\text{corr}(f_C, \text{MOD}_3) \geq 0.9$ and $E(C) < 1/(10 \cdot \log^2(s+1))$ falsifies (i); a random-wiring trial with $E(C') > \log(s+1)$ falsifies (ii).

Rationale (proposer’s reasoning):

Bounded arithmetic / Cook-Reckhow-Krajíček separates $V^0[2]$ from $V^0[2, \oplus_3]$ via counting witnesses that respect the parity vs mod-3 structural divide; the out-degree sequence of any $ACC^0[2]$ circuit on MOD_3 must absorb mod-3 fan-out collisions that cannot be balanced by XOR-only mod-2 arithmetic, forcing additive structure (small Ruzsa doubling, equivalently large E). Conversely random $ACC^0[2]$ wiring produces out-degrees that approximate a Poisson-distributed Sidon-like multiset whose E concentrates at the heuristic-random $\Theta(s^{\{-1/2\}})$ scale (Plünnecke baseline for fresh independent integers). The invariant lives on the DAG STRUCTURE (not the truth table), uses non-polynomial counting (not F_q -extendable), avoiding NATURAL_PROOFS and ALGEBRIZATION while staying within Krajíček’s structural-witness program.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68c56064b1e44a5f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 50 runs over $n \in \{6, 7, 8\}$, $s \in \{12, 20, 28, 36\}$: (i) every MOD_3 -correlated circuit (ensemble B) must satisfy $E(C) \cdot \log^2(s+1) \geq 0.1$; (ii) $\geq 90\%$ of random circuits (ensemble A) per seed must satisfy $E(C) \cdot s^{\{-1/2\}} / \log(s+1) \leq 1$; and pooled Pearson $r(MOD_3\text{-corr}, \log(E \cdot \log^2 s)) \geq +0.4$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - additive energy circuit lower bound ACC - additive combinatorics Williams algorithmic method NEXP ACC - out-degree sequence circuit complexity MOD_3 lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
```

```

from collections import defaultdict

def generate_random_circuit(n, s):
    circuit = []
    for _ in range(s):
        gate_type = random.choice(['AND', 'OR', 'NOT', 'MOD_2'])
        if gate_type == 'NOT':
            inputs = [random.randint(0, len(circuit) - 1)] if circuit else []
        else:
            min_inputs = 2 if gate_type in ['AND', 'OR', 'MOD_2'] else 1
            max_inputs = min(4, len(circuit)) if circuit else 0
            if max_inputs < min_inputs:
                inputs = []
            else:
                num_inputs = random.randint(min_inputs, max_inputs)
                inputs = random.sample(range(len(circuit)), num_inputs) if circuit else []
        circuit.append({'type': gate_type, 'inputs': inputs})
    return circuit

def compute_out_degrees(circuit):
    out_degrees = [0] * len(circuit)
    for gate in circuit:
        for input_idx in gate['inputs']:
            out_degrees[input_idx] += 1
    return out_degrees

def compute_additive_energy(out_degrees):
    s = len(out_degrees)
    if s == 0:
        return 0.0
    sum_counts = defaultdict(int)
    for i, j in itertools.product(range(s), repeat=2):
        sum_counts[out_degrees[i] + out_degrees[j]] += 1
    energy = 0
    for count in sum_counts.values():
        energy += count * count
    return energy / (s * s * s)

def compute_correlation(circuit, n, target_func):
    if n > 8:
        return 0.0
    inputs = list(itertools.product([0, 1], repeat=n))
    corr = 0.0
    for input_tuple in inputs:
        output = evaluate_circuit(circuit, input_tuple)
        corr += output * target_func(input_tuple)
    corr /= (2 ** n)
    return corr

def evaluate_circuit(circuit, input_tuple):
    values = list(input_tuple)
    for gate in circuit:
        if gate['type'] == 'NOT':
            values.append(1 - values[gate['inputs'][0]])
        elif gate['type'] == 'AND':
            val = 1

```

```

        for idx in gate['inputs']:
            val &= values[idx]
        values.append(val)
    elif gate['type'] == 'OR':
        val = 0
        for idx in gate['inputs']:
            val |= values[idx]
        values.append(val)
    elif gate['type'] == 'MOD_2':
        val = sum(values[idx] for idx in gate['inputs']) % 2
        values.append(val)
    return values[-1] if values else 0

def run_trial(seed):
    random.seed(seed)
    n = random.choice([6, 7, 8])
    s = random.choice([12, 20, 28, 36])
    circuits = []
    for _ in range(50):
        circuit = generate_random_circuit(n, s)
        circuits.append(circuit)
    mod3_circuits = []
    and_circuits = []
    for circuit in circuits:
        corr_mod3 = compute_correlation(circuit, n, lambda x: sum(x) % 3)
        corr_and = compute_correlation(circuit, n, lambda x: all(x))
        if corr_mod3 >= 0.9:
            mod3_circuits.append(circuit)
        if corr_and >= 0.9:
            and_circuits.append(circuit)
    if not mod3_circuits:
        return {
            "metric_name": "E(C)*log2(s+1)",
            "metric_value": 0.0,
            "instances_tested": len(circuits),
            "conjecture_holds": False,
            "counterexample": "No MOD_3-correlated circuits found"
        }
    min_mod3_energy = float('inf')
    for circuit in mod3_circuits:
        out_degrees = compute_out_degrees(circuit)
        energy = compute_additive_energy(out_degrees)
        min_mod3_energy = min(min_mod3_energy, energy * math.log(s + 1) ** 2)
    random_energy_values = []
    for circuit in circuits:
        out_degrees = compute_out_degrees(circuit)
        energy = compute_additive_energy(out_degrees)
        random_energy_values.append(energy * math.sqrt(s) / math.log(s + 1))
    fraction_random = sum(1 for x in random_energy_values if x <= 1) / len(random_energy_values)
    corr_values = []
    energy_values = []
    for circuit in circuits + mod3_circuits + and_circuits:
        corr_mod3 = compute_correlation(circuit, n, lambda x: sum(x) % 3)
        out_degrees = compute_out_degrees(circuit)
        energy = compute_additive_energy(out_degrees)
        corr_values.append(corr_mod3)

```

```

        energy_values.append(math.log(energy * math.log(s + 1) ** 2))
    pearson_r = compute_pearson(corr_values, energy_values)
    conjecture_holds = (min_mod3_energy >= 0.1) and (fraction_random >= 0.9) and (pearson_r >= 0.4)
    counterexample = ""
    if not conjecture_holds:
        if min_mod3_energy < 0.1:
            counterexample = f"MOD_3-correlated circuit with  $E(C) \cdot \log^2(s+1) = \{min\_mod3\_energy\}$ "
        elif fraction_random < 0.9:
            counterexample = f"Random circuit with  $E(C) \cdot s^{1/2} / \log(s+1) > 1$ "
        else:
            counterexample = f"Pearson  $r = \{pearson\_r\} < 0.4$ "
    return {
        "metric_name": "E(C) * log2(s+1)",
        "metric_value": min_mod3_energy,
        "instances_tested": len(circuits),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def compute_pearson(x, y):
    n = len(x)
    if n == 0:
        return 0.0
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / n
    std_x = math.sqrt(sum((xi - mean_x) ** 2 for xi in x) / n)
    std_y = math.sqrt(sum((yi - mean_y) ** 2 for yi in y) / n)
    if std_x == 0 or std_y == 0:
        return 0.0
    return cov / (std_x * std_y)

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        trials.append(trial)
    metric_values = [trial["metric_value"] for trial in trials]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)
    if all(trial["conjecture_holds"] for trial in trials):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not trial["conjecture_holds"] for trial in trials):
        first_failing_seed = next(seed for seed, trial in zip(seeds, trials) if not trial["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={trials[seeds.index(first_failing_seed)]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE reason=all_trials_failed")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb0cda4.py", line 162, in <module>
    trial = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb0cda4.py", line 94, in run_trial
    corr_mod3 = compute_correlation(circuit, n, lambda x: sum(x) % 3)
                ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb0cda4.py", line 58, in compute_correlation
    output = evaluate_circuit(circuit, input_tuple)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdb0cda4.py", line 67, in evaluate_circuit
    values.append(1 - values[gate['inputs']][0])
                    ~~~~~^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in `evaluate_circuit` (NOT gate referencing an out-of-range input index) before any data was produced, so neither the support nor falsification conditions could be evaluated. | next: Fix the circuit generator/evaluator so NOT gates only reference already-defined gate indices (validate topological order and input bounds), then re-run the 30×50 trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	198338
2	preregistration	claude_max	opus	0	0	9213
3	novelty	claude_max	opus	0	0	2957
4	novelty	claude_max	opus	0	0	6970
5	test_gen	mistral	codestral-latest	0	0	321443
6	test_gen	mistral	codestral-latest	0	0	317226
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	221456
8	test_gen	mistral	codestral-latest	0	0	319867
9	judge	claude_max	opus	0	0	4237

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1401707 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/7e5a5fe0905f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7e5a5fe0905f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7e5a5fe0905f.tar.gz` (if generated)